

Initialising a Variable-Speed Microturbine Model

A walkthrough of the initialization script `data_v2.m`

The plant is a variable-speed, single-shaft, recuperated microturbine running in *electrical priority*, modelled in Simulink (`microturbine_model_v2.slx`). Before every simulation the script `data_v2.m` runs once and does three jobs: it declares all physical and geometric parameters, fixes the machine's hardware constants — above all the turbine choke constant K_{choke} — and computes a fully self-consistent steady-state operating point that seeds every integrator, together with the optimal-speed schedule the governor follows. My target was a **flat start**: the model must open exactly at its steady operating point, with no spurious startup transient.

Design philosophy

Three principles shape the whole script:

- **One source of truth for the hardware.** The turbine throat is captured by a single constant, $K_{choke} = m_{eg} \cdot \sqrt{T_3} / P_3$, evaluated once at the nominal design point and never recomputed. The plant and every offline calculation use that same number.
- **Shared physics.** The offline design calculations and the Simulink plant must describe the identical machine, so I factored the thermodynamics into small shared engines (`recupSteady5`, `solvePoint`) that every routine calls. The preliminary design point then reproduces the plant's own equilibrium to machine precision.
- **Flat start.** Each integrator's initial condition (shaft speed, plenum pressure, recuperator metal, fuel) is the *closed-loop equilibrium* the governor will hold at the design load — so the machine starts, and stays, at steady state.

What the script does, in order

0 Workspace & parameters. It clears the workspace (keeping only an optional app-supplied ambient temperature `T1_app`), adds the helper folders to the path, and declares the thermodynamic and geometric constants: pressures, specific heats, compressor/turbine/mechanical/generator efficiencies, the lumped duct+recuperator+combustor pressure-drop coefficient, LHV, shaft inertia, and the heat-recovery-boiler data.

1 Compressor map. `drawCompressorMap` turns the raw map (`CompressorMap.mat`) into the interpolation data used by both the code and the plant. It stacks the five speed lines into scattered vectors (mass flow, pressure ratio, speed in rad/s), removes near-duplicate points, publishes those vectors and the surge line to the workspace, and draws the map. These are the exact scattered data behind the plant's compressor lookup table.

`removeDuplicates` — a utility it calls to drop points that share the same (pressure ratio, speed) within a tolerance, so the scattered interpolant stays single-valued.

2 Design guess & design air flow. I set a first guess for the design conditions ($PR = 4$, $n = 65,000$ rpm, $T_3 = 950$ °C) and the recuperator hardware constants, then read the design air flow at that (PR , n) from the map.

`designAirFlowRate.slx` — a small Simscape harness that evaluates the compressor map at the guessed (PR , n) and returns the air mass flow, using the same lookup as the plant's compressor so the design flow is map-consistent.

3 Fixed hardware constant K_{choke} . The design data are bundled into a struct and `computeKchoke` returns the turbine choke constant, evaluated at the nominal 15 °C. From here on it is treated as fixed hardware.

`computeKchoke` — deliberately self-contained: it computes *only* what the constant needs — the turbine-inlet pressure P_3 and the exhaust-gas flow m_{eg} (found by closing the combustor energy balance through the recuperator) — and returns $K_{choke} = m_{eg} \cdot \sqrt{T_3} / P_3$. It does not build the full cycle, so the code reads as exactly what it is: the definition of the turbine throat.

4 First-guess cycle. A convenience handle `dpAt` wraps `designPoint` with the fixed K_{choke} and the operating temperature. Calling it at the guess gives a complete cycle and, in particular, the design electrical power P_0 .

`designPoint` — given (PR , speed, T_3 , air flow) and the fixed K_{choke} , it solves the full steady cycle: compressor outlet, plenum pressure, recuperator, combustor energy balance, choked turbine and exhaust back-pressure. It returns every station (T , P , mass flow, enthalpy, entropy), the recuperator metal-temperature profile, compressor/turbine powers, net electrical power and efficiency, the shaft kinetic energy and the fuel flow — precisely the initial-condition fields the Simulink integrators expect. Stations are assembled by the helper `makeStation`, which packs one thermodynamic point (T , P , enthalpy, entropy, density, mass flow, fluid type) into a tidy struct.

5 Optimal-speed schedule. `computeSpeedSchedule` builds the map `n_ref(P_el)` that the variable-speed governor follows, using the fixed `K_choke` and the map vectors from step 1.

`computeSpeedSchedule` — it sweeps a grid of (speed, pressure ratio), solves each steady point with the plant physics, and keeps only feasible ones: inside the speed range (with margin), turbine still choked, compressor surge margin respected, and turbine-inlet temperature within its working window and above the flame-stability floor. For every power level it selects the speed that *maximises electrical efficiency*, smooths that into a monotonic schedule, and packages a ready-to-use 1-D lookup for the governor (it also returns the map interpolant for the next step).

6 Flat-start operating point. `scheduleIC` returns the precise (speed, pressure ratio, T3) at which the closed loop settles for the design load `P0`.

`scheduleIC` — the governor will hold `n_ref(P0)`, so the only remaining freedom is the pressure ratio. It takes the speed straight from the schedule lookup (identical to the Simulink block) and solves the single pressure ratio that makes the plant produce *exactly* `P0` at that speed. The result is the true closed-loop equilibrium — seeding the integrators from it removes the startup transient entirely.

7 Final cycle & visual check. `dpAt` is called once more at the precise (PR, speed, T3); this `dp` struct carries the initial conditions for the whole model. `plotCycle` then draws the temperature–entropy diagram of the cycle (compression, recuperator+combustor heat addition, expansion) as a quick physical sanity check.

8 Derived design quantities. The script sets the turbine-temperature protection limit, sizes the heat-recovery-boiler design water flow from the turbine-exit conditions, and copies the recuperator hardware constants into `dp`.

9 Housekeeping. Finally it loads the model and caps every scope's data buffer (decimation) so the long, fine-step real-time runs stay responsive, then reports that initialization succeeded.

The two shared physics engines

`recupSteady5` — the five-node counter-flow recuperator with metal thermal mass at steady state. It solves the node metal temperatures (a small linear system) and returns the cold- and hot-side outlet temperatures. It is the *same* recuperator model the plant runs, and it is called by `designPoint`, `computeKchoke` and `solvePoint` — which is what keeps them all consistent.

`solvePoint` — the steady operating-point solver shared by the schedule and by `scheduleIC`. Given a speed, a pressure ratio and the fixed `K_choke`, it derives T3 from the choke relation and closes the combustor balance through `recupSteady5`, returning power, efficiency, temperatures and flows. Factoring it out is what makes the offline schedule and the plant numerically identical.

The pay-off of this structure is that the machine is described by one consistent set of physics and one fixed hardware constant; the offline design point reproduces the plant's own equilibrium to machine precision; and the simulation opens flat at any chosen load and ambient temperature — a clean, debuggable foundation for the control layer built on top.